# CHƯƠNG 2: ĐẠI SỐ BOOLE VÀ CÁC CỔNG LOGIC CƠ BẢN

**MÔN HỌC: KỸ THUẬT SỐ — HỌC KỲ I**

## Mục tiêu chương học:

- Nắm vững các tiên đề Huntington, định lý De Morgan và tính chất đại số Boole.
- Hiểu sâu nguyên lý, bảng chân lý và phương trình của 7 cổng logic cơ bản.
- Thành thạo phương pháp sử dụng cổng vạn năng NAND và NOR để thiết kế mạch logic.
- Phân biệt và chuyển đổi giữa dạng Tổng các Tích chuẩn tắc (SOP) và Tích các Tổng (POS).
- Tối thiểu hóa hàm logic đa biến bằng phương pháp Bìa Karnaugh (K-Map 2, 3, 4 biến) có điều kiện tùy định.

```
CÔNG THỨC:
F(A, B, C) = Σ m(...) + d(...) <===> Tối thiểu hóa: K-Map 16 ô
Trọng tâm thiết kế mạch logic tối ưu phần cứng
```

# 2.1. CÁC TIÊN ĐỀ VÀ ĐỊNH LÝ CỐT LÕI CỦA ĐẠI SỐ BOOLE

**CHƯƠNG 2: CƠ SỞ ĐẠI SỐ BOOLE**

## Bảng đối ngẫu các định lý đại số Boole:

| Tên định lý | Dạng toán tử HOẶC (+ / OR) | Dạng toán tử VÀ (· / AND) |
|---|---|---|
| Phần tử trung hòa | A + 0 = A | A · 1 = A |
| Phần tử bù | A + A' = 1 | A · A' = 0 |
| Giao hoán | A + B = B + A | A · B = B · A |
| Kết hợp | (A + B) + C = A + (B + C) | (A · B) · C = A · (B · C) |
| Phân phối | A · (B + C) = AB + AC | A + (B · C) = (A + B)(A + C) |
| Hấp thụ (Absorption) | A + A · B = A | A · (A + B) = A |
| Đồng thuận (Consensus) | AB + A'C + BC = AB + A'C | (A+B)(A'+C)(B+C) = (A+B)(A'+C) |

# 2.2. ĐỊNH LÝ DE MORGAN VÀ Ý NGHĨA KỸ THUẬT

CHƯƠNG 2: ĐỊNH LÝ BOOLE

## Hai mệnh đề của Định lý De Morgan:

- Mệnh đề 1: Phần bù của một tích logic bằng tổng các phần bù của từng biến.
- Mệnh đề 2: Phần bù của một tổng logic bằng tích các phần bù của từng biến.

CÔNG THỨC:
Mệnh đề 1: NOT (A · B) = NOT A + NOT B [Cổng NAND tương đương Cổng OR ngõ vào đảo] Mệnh đề 2: NOT (A + B) = NOT A · NOT B [Cổng NOR tương đương Cổng AND ngõ vào đảo] Tổng quát n biến: NOT (x1 · x2 · ... · xn) = NOT x1 + NOT x2 + ... + NOT xn NOT (x1 + x2 + ... + xn) = NOT x1 · NOT x2 · ... · NOT xn
Cơ sở toán học để chuyển đổi qua lại giữa các cổng logic trong công nghệ chế tạo chip

# 2.3. BẢNG TỔNG HỢP 7 CỔNG LOGIC CƠ BẢN

**CHƯƠNG 2: CỔNG LOGIC**

### Phương trình và bảng chân lý của các cổng logic:

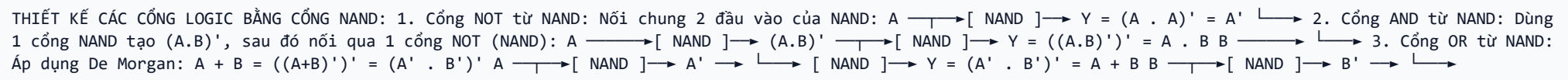
| Tên cổng | Ký hiệu mạch | Phương trình ngõ ra Y | Bảng chân lý (A B → Y) |
|---|---|---|---|
| AND | Cổng D | Y = A · B | 00→0, 01→0, 10→0, 11→1 |
| OR | Cổng cong nhọn | Y = A + B | 00→0, 01→1, 10→1, 11→1 |
| NOT | Tam giác + bong bóng | Y = A' | 0→1, 1→0 |
| NAND | AND + bong bóng | Y = (A · B)' | 00→1, 01→1, 10→1, 11→0 |
| NOR | OR + bong bóng | Y = (A + B)' | 00→1, 01→0, 10→0, 11→0 |
| XOR | Cong đôi (Độc quyền) | Y = A □ B = A'B + AB' | 00→0, 01→1, 10→1, 11→0 (Khác nhau thì 1) |
| XNOR | XOR + bong bóng | Y = (A □ B)' = AB + A'B' | 00→1, 01→0, 10→0, 11→1 (Bằng nhau thì 1) |

# 2.4. CỔNG LOGIC VẠN NĂNG (UNIVERSAL GATES: NAND & NOR)

**CHƯƠNG 2: THIẾT KẾ PHẦN CỨNG**

## Tại sao gọi NAND và NOR là cổng vạn năng?

- Chỉ cần dùng duy nhất một loại cổng NAND (hoặc chỉ dùng NOR), ta có thể tạo ra tất cả các hàm logic cơ bản: NOT, AND, OR, XOR.
- Lợi ích sản xuất: Giảm giá thành chip, dễ sản xuất hàng loạt bằng công nghệ CMOS/TTL (cổng NAND trong CMOS tiêu tốn ít bóng bán dẫn nhất).




Sơ đồ / Bảng: Sơ đồ chuyển đổi tương đương mạch logic chỉ sử dụng cổng NAND 2 đầu vào

# 2.5. HAI DẠNG BIỂU DIỄN CHUẨN TẮC: SOP VÀ POS

CHƯƠNG 2: DẠNG CHUẨN TẮC

## Định nghĩa và Ký hiệu toán học:

**1. Dạng Tổng các Tích chuẩn tắc (Sum-of-Products - SOP / Minterm):**

- Mỗi Minterm m_i là một tích chứa đầy đủ tất cả các biến, nhận giá trị 1 tại đúng một hàng của bảng chân lý.
- Hàm logic F được viết là tổng các Minterm mà tại đó F = 1: F = Σ m(...).

**2. Dạng Tích các Tổng chuẩn tắc (Product-of-Sums - POS / Maxterm):**

- Mỗi Maxterm M_i là một tổng chứa đầy đủ tất cả các biến, nhận giá trị 0 tại đúng một hàng của bảng chân lý.
- Hàm logic F được viết là tích các Maxterm mà tại đó F = 0: F = Π M(...).

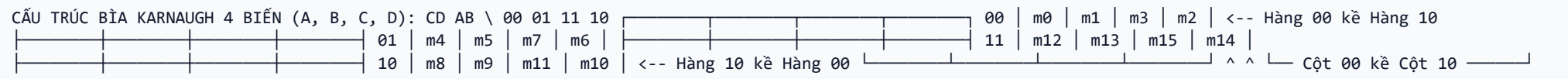
**CÔNG THỨC:**
**Mối quan hệ bù giữa Minterm và Maxterm: m_i = NOT (M_i) và M_i = NOT (m_i) Ví dụ: Cho hàm 3 biến F(A, B, C) có bảng chân lý nhận giá trị 1 tại các hàng 1, 4, 5, 7: - Dạng SOP: F = Σ m(1, 4, 5, 7) = A'B'C + AB'C' + AB'C + ABC - Dạng POS: F = Π M(0, 2, 3, 6) = (A+B+C)(A+B'+C)(A+B'+C')(A'+B'+C)**
**Dạng SOP thích hợp với mạch AND-OR; dạng POS thích hợp với mạch OR-AND**

# 2.6. PHƯƠNG PHÁP TỐI THIỂU HÓA HÀM LOGIC BẰNG BÌA KARNAUGH

**CHƯƠNG 2: BÌA KARNAUGH (K-MAP)**

### Bản chất hình học của Bìa Karnaugh:

- Bìa Karnaugh là một bảng chứa 2^n ô (với n là số biến logic). Mỗi ô đại diện cho một Minterm.
- Quy tắc mã Gray: Các hàng và cột liền kề chỉ được sai khác nhau đúng 1 bit (00 → 01 → 11 → 10). Nhờ đó, hai ô cạnh nhau sẽ triệt tiêu được 1 biến: x.y + x.y' = x.
- Tính chất gập mép hình xuyến (Torus): Ô ở cột đầu tiên kề với ô ở cột cuối cùng; ô ở hàng đầu tiên kề với ô ở hàng cuối cùng.




Sơ đồ / Bảng: Thứ tự sắp xếp các ô Minterm theo mã Gray trong K-Map 4 biến

# 2.7. QUY TẮC GOM NHÓM K-MAP VÀ ĐIỀU KIỆN TÙY ĐỊNH (DON'T-CARE)

**CHƯƠNG 2: BÌA KARNAUGH (K-MAP)**

### Bốn nguyên tắc gom nhóm tối ưu:

1. Kích thước nhóm phải là lũy thừa của 2: nhóm 1, 2, 4, 8, hoặc 16 ô chứa số 1.
2. Ưu tiên tạo nhóm có kích thước lớn nhất có thể để triệt tiêu tối đa số biến (nhóm 2^k ô triệt tiêu k biến).
3. Số lượng nhóm phải là ít nhất sao cho phủ kín toàn bộ các số 1 trên bìa.
4. Điều kiện tùy định (Don't-Care - ký hiệu d hoặc X): Là các tổ hợp ngõ vào không bao giờ xảy ra hoặc ngõ ra không quan trọng (ví dụ các mã từ 10 đến 15 trong mã BCD). Ta có thể coi X = 1 nếu giúp mở rộng nhóm, hoặc coi X = 0 nếu không cần gom.

**CÔNG THỨC:**
**Số biến còn lại sau khi gom nhóm K-Map: - Nhóm 1 ô: Giữ nguyên cả 4 biến (ví dụ: A'BCD) - Nhóm 2 ô: Triệt tiêu 1 biến, còn lại 3 biến (ví dụ: A'BC) - Nhóm 4 ô: Triệt tiêu 2 biến, còn lại 2 biến (ví dụ: AC) - Nhóm 8 ô: Triệt tiêu 3 biến, còn lại 1 biến (ví dụ: A) - Nhóm 16 ô: Triệt tiêu toàn bộ biến, F = 1**
**Quy tắc vàng: Nhóm càng to thì hàm logic càng ngắn và mạch càng ít cổng**

# 2.8. VÍ DỤ TỪNG BƯỚC: TỐI THIỂU HÓA HÀM 4 BIẾN CÓ DON'T CARE

**CHƯƠNG 2: VÍ DỤ THỰC HÀNH**

## Đề bài: Tối thiểu hóa hàm F(A, B, C, D) = Σ m(1, 3, 7, 11, 15) + d(0, 2, 5)

```
ĐIỀN VÀO BÌA KARNAUGH 4 BIẾN: CD AB \ 00 01 11 10 ┌────────┬────────┬────────┬────────┐ 00 | X(0) | 1(1) | 1(3) | X(2) | <-- Nhóm 1: Gom cả hàng 00 thành nhóm 4 ô (X=1) ├────────┼────────┼────────┼────────┤ 01 | 0(4) | X(5) | 1(7) | 0(6) | ├────────┼────────┼────────┼────────┤ 11 | 0(12)| 0(13)| 1(15)| 0(14)| ├────────┼────────┼────────┼────────┤ 10 | 0(8) | 0(9) | 1(11)| 0(10)| └────────┴────────┴────────┴────────┘ ^ └── Nhóm 2: Cột CD = 11 (gồm m3, m7, m15, m11) KẾT QUẢ GOM NHÓM: - Nhóm 1 (Hàng AB=00: m0, m1, m3, m2): Biến A=0, B=0 không đổi; C và D triệt tiêu --> Số hạng: A'B' - Nhóm 2 (Cột CD=11: m3, m7, m15, m11): C=1, D=1 không đổi; A và B triệt tiêu --> Số hạng: C.D ===> HÀM TỐI TIỂU CUỐI CÙNG: F(A, B, C, D) = A'B' + CD
```

Sơ đồ / Bảng: Cách tận dụng Don't-Care m0, m2 để tạo nhóm 4 ô thay vì nhóm 2 ô rời rạc

# 2.9. BÀI TẬP VÀ CÂU HỎI TRẮC NGHIỆM ÔN TẬP CHƯƠNG 2

## CHƯƠNG 2: ĐÁNH GIÁ KIẾN THỨC

Câu hỏi trắc nghiệm 1: Theo định lý De Morgan, biểu thức logic NOT (A + B + C) tương đương với biểu thức nào?

A. A' + B' + C'  B. A' . B' . C'
C. (A . B . C)'  D. A . B . C

Đáp án đúng: B — Định lý De Morgan mở rộng: Phủ định của một tổng bằng tích các phủ định: (A+B+C)' = A' . B' . C'.

Câu hỏi trắc nghiệm 2: Cổng logic nào có ngõ ra bằng 1 khi và chỉ khi hai ngõ vào có giá trị khác nhau?

A. Cổng AND  B. Cổng OR
C. Cổng XOR (Exclusive-OR)  D. Cổng XNOR

Đáp án đúng: C — Bảng chân lý cổng XOR: 0 XOR 0 = 0, 1 XOR 1 = 0, 0 XOR 1 = 1, 1 XOR 0 = 1. Ngõ ra chỉ bằng 1 khi hai ngõ vào khác mức logic.

# 2.9. BÀI TẬP VÀ CÂU HỎI TRẮC NGHIỆM ÔN TẬP (TIẾP)

## CHƯƠNG 2: ĐÁNH GIÁ KIẾN THỨC

Câu hỏi trắc nghiệm 3: Trong bìa Karnaugh 4 biến, một nhóm gom gồm 8 ô liền kề chứa số 1 sẽ rút gọn được bao nhiêu biến?

A. 1 biến  B. 2 biến
C. 3 biến  D. 4 biến

Đáp án đúng: C — Nhóm 2^k ô triệt tiêu được k biến. Ở đây nhóm 8 ô = 2^3 ô, do đó triệt tiêu được 3 biến, số hạng chỉ còn lại đúng 1 biến duy nhất.

Câu hỏi trắc nghiệm 4: Điều kiện tùy định (Don't-Care conditions) trong bìa Karnaugh được tận dụng như thế nào?

A. Bắt buộc phải gom tất cả các ô tùy định vào nhóm.  B. Không được phép gom các ô tùy định vào bất kỳ nhóm nào.
C. Có thể coi là 1 nếu giúp nhóm lớn hơn, hoặc coi là 0 nếu không cần thiết.  D. Chỉ dùng khi bảng chân lý có số biến lẻ.

Đáp án đúng: C — Ô Don't Care có giá trị tùy ý, ta linh hoạt gán bằng 1 để tạo nhóm 2^k lớn hơn nhằm tối giản hàm tối đa, các ô Don't Care còn lại coi như bằng 0.